

Comparativo de Desempenho ETL (Pandas vs. PySpark)

1. Objetivo do Projeto

Desenvolver e documentar um experimento rigoroso para a disciplina de Sistemas Distribuídos, avaliando a escalabilidade e o consumo de recursos de hardware (Tempo, CPU e RAM) entre uma abordagem de processamento sequencial tradicional e uma arquitetura distribuída sob diferentes volumes de dados. O projeto visa embasar o relatório final que será submetido ao Prof. Valêncio.

2. Metodologia e Preparação do Ambiente

Para garantir a lisura científica, o isolamento de variáveis e um comparativo de hardware justo, ambos os *pipelines* foram executados fisicamente na mesma máquina.

- **Infraestrutura Nativa (WSL 2):** A arquitetura inicial baseada puramente no sistema operacional Windows apresentou gargalos de I/O e incompatibilidades de permissões intrínsecas ao ecossistema Hadoop/Spark (`winutils.exe` e `hadoop.dll`). Como solução metodológica, o ambiente de testes foi migrado para o **Windows Subsystem for Linux (WSL 2 - Ubuntu)**. Essa alteração foi crucial para permitir que o Apache Spark rodasse de forma estável, garantindo medições confiáveis.
- **Isolamento de Dependências:** Todo o código foi executado sob um ambiente virtual Linux (`venv_linux`) rodando o Java 11 e o PySpark em sua versão estável (3.5.1), garantindo reprodutibilidade do experimento.
- **Geração de Dados Sintéticos (`00 gerar_dados.py`):** Criamos um gerador de dados automatizado utilizando a biblioteca `Faker` e matrizes em `numpy` para alta performance vetorial. O modelo gerou arquivos CSV simulando um modelo estrela clássico: *Clientes* (Dimensão), *Produtos* (Dimensão) e *Pedidos* (Tabela Fato).

3. Implementação e Coleta de Métricas (O "Como")

A mensuração do impacto computacional foi o ponto central da metodologia adotada.

- **Profiler de Monitoramento Customizado (`profiler.py`):** Foi desenvolvido um decorador em Python utilizando as bibliotecas `psutil` e `threading`. Durante a execução completa de cada ETL, uma *thread* rodou em segundo plano a cada 50 milissegundos, registrando sistematicamente o tempo cronometrado, o pico exato de processamento (CPU) e a alocação máxima em memória (RAM).
- **A Abordagem Sequencial (`01_etl_pandas.py`):** Desenvolvemos um *pipeline* utilizando a biblioteca Pandas. O fluxo completo (extração, cruzamento de dados, criação de coluna derivada de faturamento e agregação por estado/categoria) dependeu do carregamento total do *dataset* para a memória RAM física antes da escrita no disco.
- **A Abordagem Distribuída (`02_etl_pyspark.py`):** Instanciamos uma `SparkSession` utilizando a diretiva `master("local[*]")`, que instrui o motor distribuído a fatiar o processamento através de partições em todos os núcleos lógicos da máquina. Para fins de paridade com a saída do Pandas, o Grafo Acíclico Direcionado (DAG) do Spark foi forçado a unificar o resultado em um único *node* final de escrita usando o método `.coalesce(1)`.

4. Bateria de Testes (Escalonamento Vertical)

Para validar o comportamento do hardware, os dados foram submetidos aos dois *pipelines* sob três cenários incrementais de estresse:

1. **Cenário A:** 100 mil registros (Volume Leve)
2. **Cenário B:** 1 milhão de registros (Volume Intermediário)
3. **Cenário C:** 5 milhões de registros (Carga de Estresse)

5. Discussão Primária dos Resultados

As métricas coletadas revelaram distinções claras entre as arquiteturas:

- **Gargalo de Memória vs. Estabilidade:** O Pandas demonstrou ser extremamente rápido para os cenários iniciais, porém seu consumo de RAM obedeceu a um crescimento escalar severo (alcançando o pico de **916 MB** nos 5 milhões de linhas), provando sua limitação e propensão ao erro de *Out*

of Memory em casos de *Big Data*. O PySpark compensou isso isolando os lotes na JVM, resultando em um consumo do Driver Python fixado em seguros **~36 MB** em todos os cenários.

- **Overhead do Ecossistema:** O Spark obteve tempos maiores nos testes (oscilando entre 18s e 21s, independentemente da carga). Isso comprova teoricamente o elevado custo de *overhead* inicial do ambiente distribuído (subida da JVM, cálculo do DAG e alocação de executores), deixando claro que a tecnologia distribuída compensa apenas quando a volumetria supera a velocidade da RAM física disponível na máquina *single-node*.

6. Geração de Artefatos Visuais

Os resultados textuais foram automatizados através de um ambiente Jupyter (`analise_graficos.ipynb`), devidamente renderizado no Linux com a integração `ipykernel` . Este bloco final gerou *plots* visuais (Matplotlib/Seaborn) sobrepondo o consumo dos dois métodos, fornecendo as figuras necessárias para a demonstração gráfica no corpo do relatório final da faculdade.